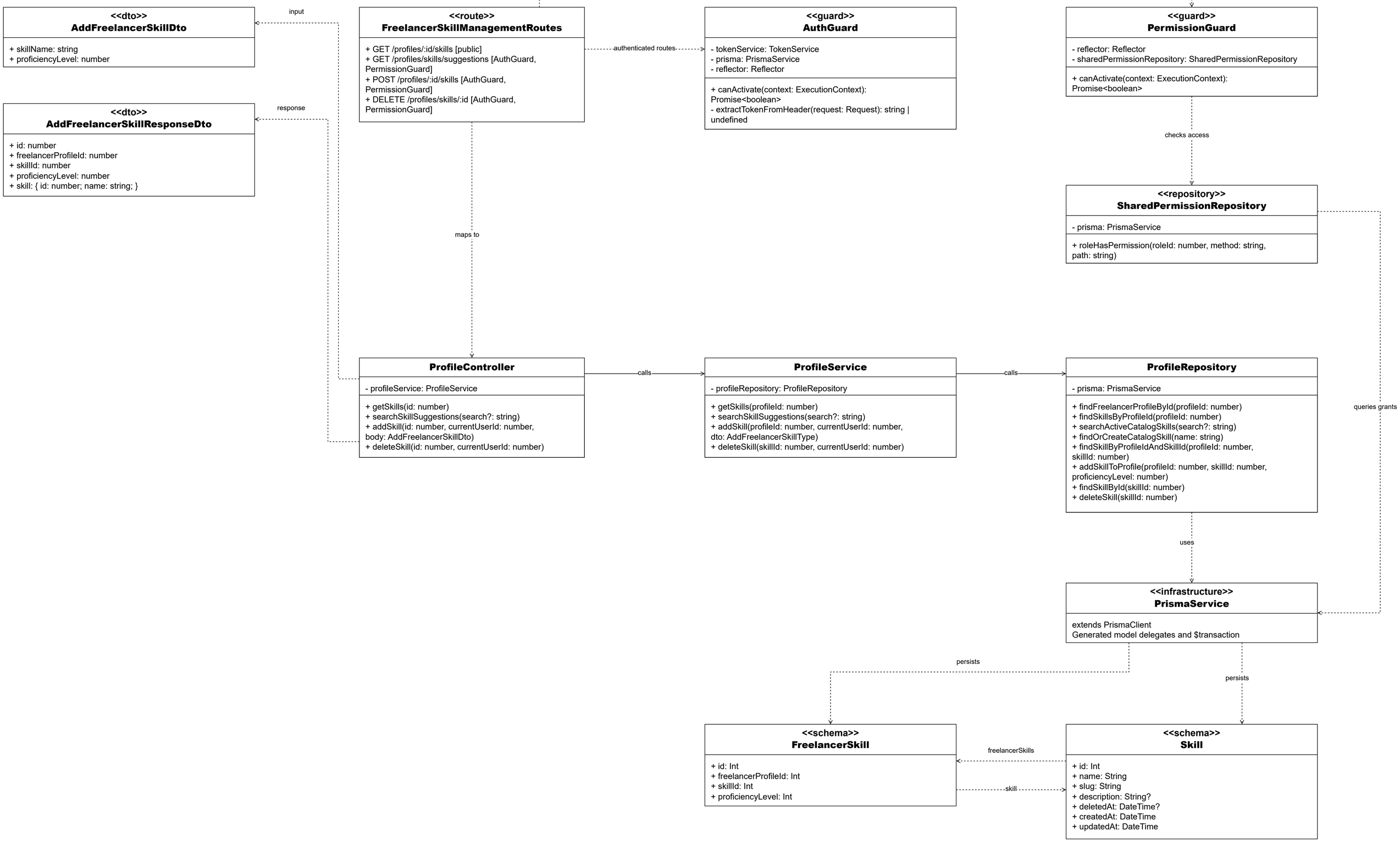

<<dto>>
AddFreelancerSkillDto
+ skillName: string
+ proficiencyLevel: number
input
<<dto>>
AddFreelancerSkillResponseDto
+ id: number
+ freelancerProfileId: number
+ skillId: number
+ proficiencyLevel: number
+ skill: { id: number; name: string; }
response
<<route>>
FreelancerSkillManagementRoutes
+ GET /profiles/:id/skills [public]
+ GET /profiles/skills/suggestions [AuthGuard, PermissionGuard]
+ POST /profiles/:id/skills [AuthGuard, PermissionGuard]
+ DELETE /profiles/skills/:id [AuthGuard, PermissionGuard]
authenticated routes
authenticated routes
<<guard>>
AuthGuard
- tokenService: TokenService
- prisma: PrismaService
- reflector: Reflector
+ canActivate(context: ExecutionContext): Promise<boolean>
- extractTokenFromHeader(request: Request): string | undefined
<<guard>>
PermissionGuard
- reflector: Reflector
- sharedPermissionRepository: SharedPermissionRepository
+ canActivate(context: ExecutionContext): Promise<boolean>
checks access
<<repository>>
SharedPermissionRepository
- prisma: PrismaService
+ roleHasPermission(roleId: number, method: string, path: string)
maps to
ProfileController
- profileService: ProfileService
+ getSkills(id: number)
+ searchSkillSuggestions(search?: string)
+ addSkill(id: number, currentUserId: number, body: AddFreelancerSkillDto)
+ deleteSkill(id: number, currentUserId: number)
calls
ProfileService
- profileRepository: ProfileRepository
+ getSkills(profileId: number)
+ searchSkillSuggestions(search?: string)
+ addSkill(profileId: number, currentUserId: number, dto: AddFreelancerSkillType)
+ deleteSkill(skillId: number, currentUserId: number)
calls
ProfileRepository
- prisma: PrismaService
+ findFreelancerProfileById(profileId: number)
+ findSkillsByProfileId(profileId: number)
+ searchActiveCatalogSkills(search?: string)
+ findOrCreateCatalogSkill(name: string)
+ findSkillByProfileIdAndSkillId(profileId: number, skillId: number)
+ addSkillToProfile(profileId: number, skillId: number, proficiencyLevel: number)
+ findSkillById(skillId: number)
+ deleteSkill(skillId: number)
queries grants
uses
<<infrastructure>>
PrismaService
extends PrismaClient
Generated model delegates and $transaction
persists
persists
<<schema>>
FreelancerSkill
+ id: Int
+ freelancerProfileId: Int
+ skillId: Int
+ proficiencyLevel: Int
freelancerSkills
skill
<<schema>>
Skill
+ id: Int
+ name: String
+ slug: String
+ description: String?
+ deletedAt: DateTime?
+ createdAt: DateTime
+ updatedAt: DateTime